Nama : Irgieta Villa Cantika

NIM : 23031554107

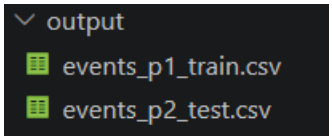
Mata Kuliah : Graf Sains Data 2023D

UTS Graf Sains Data

1. Pertanyaan Pendahuluan
   1) Database yang digunakan pada tutorial ini adalah neo4j, dapat dilihat pada nomor 3 di subbab 1.2

```
def add_friend(driver, name, friend_name):
    driver.execute_query(
        "MERGE (a:Person {name: $name}) "
        "MERGE (friend:Person {name: $friend_name}) "
        "MERGE (a)-[:KNOWS]->(friend)",
        name=name, friend_name=friend_name, database_="neo4j",)
```

   2) Perintah Chyper, seperti DDL untuk membuat constraint, DML untuk menambah atau mengubah data, serta MATCH dan RETURN untuk mencari dan menampilkan data dapat digunakan untuk mengakses database dalam satu instance Neo4j. Database lain dalam instance yang sama dapat dipilih melalui parameter 'database' pada driver, selama database tersebut tersedia dan bisa diakses.
   3) Jenis pertemanan yang dibuat adalah graf berarah tanpa bobot (directed unweighted graph) karena hubungan KNOWS dibuat dari node Person bernama Arthur -> Guinevere, Lancelot, dan Merlin. Setiap hubungan memiliki arah dari Arthur ke temannya, tetapi tidak otomatis menunjukkan hubungan sebaliknya.
2. Soal UTS
   2.1. Wrangling dan EDA
      1) P1: 2015/05/08 – 2015/05/10
         P2: 2015/05/11 – 2015/05/12
      2) Setelah dilakukan filtering, data disimpan pada folder output dengan format csv.



      3) Cypher yang digunakan untuk mengetahui komposisi subgraph
         - q_nodes = """ MATCH (n) RETURN labels(n)[0] AS jenis_simpul, count(n) AS jumlah ORDER BY jumlah DESC"""
           bagian ini digunakan untuk mencocokkan semua node, mengambil label pertamanya sebagai jenis simpul (User dan Item), lalu menghitung jumlah per jenis. Pengelompokan dilakukan otomatis oleh RETURN karena ada fungsi agregat count.

- q_rels = """MATCH ()-[r]->() RETURN type(r) AS jenis_relasi, count(r) AS jumlah ORDER BY jumlah DESC """
  bagian ini digunakan untuk mencocokkan semua relasi, dikelompokkan menurut tipenya.

Kedua hasil tersebut diurutkan dari yang terbanyak.

	jenis_simpul	jumlah
0	User	25300
1	Item	22521

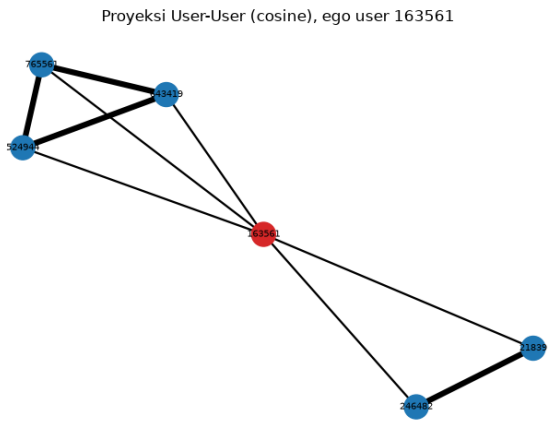
	jenis_relasi	jumlah
0	VIEWED	42890
1	ADDED_TO_CART	1123
2	BOUGHT	249

Terdapat dua jenis simpul pada graf ini, yaitu User dan Item. Simpul User berjumlah 25.300 sedangkan simpul Item berjumlah 22.521, jadi terdapat 47.821 simpul. Terdapat tiga jenis relasi, yaitu VIEWED, ADDED_TO_CART, BOUGHT dengan total relasi sebesar 44.262. Relasi VIEWED memiliki jumlah terbanyak yang menyiratkan bahwa banyak User yang melakukan kegiatan melihat (VIEWED) Item.

4) Pada VSM User, simpul User akan dilihat sebagai vector dengan dimensi Item. Bobot tiap relasi dibuat menjadi
   - VIEWED = 1
   - ADDED_TO_CART = 2
   - BOUGHT = 3

   Bobot akan dijumlahkan sesuai dengan kegiatan/relasi yang dilakukan User terhadap Item. Matriks yang sudah dibentuk akan dinormalisasi (L2) untuk digunakan dalam menghitung dot product (cosine similarity).

VSM	User
baris =	user
dimensi (kolom) =	item
shape	(25300, 22521)
nnz	35783
densitas (%)	0.00628
sparsitas (%)	99.99372
nnz/baris min	1
median	1.0
mean	1.414348
max	167
baris dengan 1 nnz (%)	82.790514



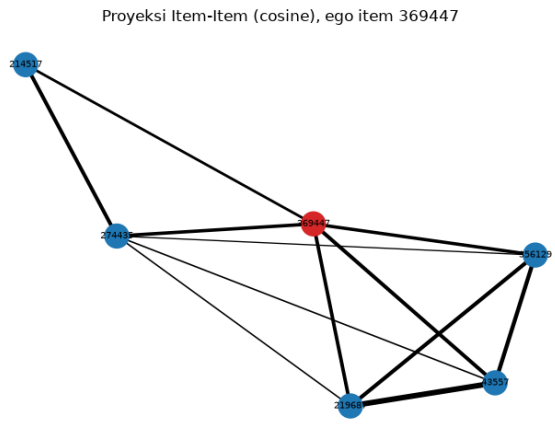
Matriks VSM User membentuk matriks X berukuran 25300 x 22521, artinya ada 25300 vektor yang masing-masing berdimensi 22521. Diperoleh *number of non-*

zero (nnz) atau jumlah sel tak nol berjumlah 35783. Nilai nnz/(25300 x 22521) menghasilkan densitas (sel yang terisi) sebesar 0.00628% dan sparsitas (sel yang kosong) 99.99372%. Sebaran jumlah item per user sangat miring: nilai minimum 1, median 1, dan rata-rata 1,41, sedangkan nilai maksimumnya 167. Sekitar 82,79% user hanya berinteraksi dengan satu item. Artinya mayoritas vektor user hanya punya satu elemen bukan nol, sehingga kemiripan (cosine) antar-user hanya dapat dihitung bila dua user mengakses item yang sama. Konsekuensinya, graf proyeksi User-User akan jarang dan banyak nilai cosine bernilai sama atau nol. Karena itu matriks disimpan dalam format sparse (csr_matrix) agar hemat memori dan komputasi.

Ego graph adalah subgraph terdiri dari satu simpul acuan, beberapa tetangga terdekat (di kasus 5 tetangga dengan cosine tertinggi), dan hubungan antar simpul-simpul tersebut. User paling aktif dipilih sebagai acuan (ego). User 163561 dipilih sebagai acuan, dan 5 tetangga yang terhubung padanya memiliki garis yang tipis. Dari kelima simpul tetangga ini terbentuk dua kelompok, tiga user (765561, 643419, 524944) saling terhubung dengan sisi tebal, dan dua user (246482 dan 21839) terhubung dengan sisi tebal juga. Kedua kelompok terhubung melalui simpul acuan. Semakin tebal sisi yang terhubung, semakin tinggi pula nilai cosinenya. Pola ini menunjukkan bahwa ego mirip dengan masing-masing kelompok melalui item yang berbeda.

5) Pada VSM Item, Item dilihat sebagai vektor dengan dimensi user. VSM Item diperoleh dari transpose matriks User × Item (X^T), sehingga bobot interaksinya sama dengan nomor 4 (VIEWED: 1, ADDED_TO_CART: 2, BOUGHT: 3), dijumlahkan bila berulang. Matriks kemudian dinormalisasi (L2) agar perkalian titik (dot product) antar-vektor sama dengan cosine similarity.

VSM	Item
baris =	item
dimensi (kolom) =	user
shape	(22521, 25300)
nnz	35783
densitas (%)	0.00628
sparsitas (%)	99.99372
nnz/baris min	1
median	1.0
mean	1.588873
max	68
baris dengan 1 nnz (%)	73.753386



Matriks VSM Item membentuk matriks X^T berukuran 22.521 × 25.300, artinya ada 22.521 vektor item yang masing-masing berdimensi 25.300 (user). Jumlah sel tak

nol (nnz) sebanyak 35.783, sehingga densitas 0,00628% dan sparsitas 99,99372%. Nilai nnz, densitas, dan sparsitas sama persis dengan VSM User karena X^T hanyalah transpose dari X; yang berbeda adalah jumlah baris, dimensi, dan sebaran nnz per baris. Pada VSM Item, jumlah user per item memiliki minimum 1, median 1, rata-rata 1,59, dan maksimum 68. Sekitar 73,75% item hanya diakses oleh satu user. Dibandingkan VSM User (82,79% baris dengan satu nnz; rata-rata 1,41; maksimum 167), vektor item sedikit lebih padat dan sebarannya tidak sejauh miring, tetapi mayoritas vektor tetap hanya memiliki satu elemen bukan nol. Artinya kemiripan antar-item hanya dapat dihitung bila dua item diakses oleh user yang sama, sehingga graf proyeksi Item-Item juga jarang.

Ego graph pada proyeksi Item-Item memakai item 369447 sebagai acuan (ego), yaitu item yang paling banyak diakses (68 user), beserta 5 item dengan cosine tertinggi terhadapnya. Graf yang terbentuk memiliki 6 simpul dan 12 sisi dari 15 kemungkinan pasangan, lebih rapat dibandingkan ego graph User-User (9 sisi). Ego terhubung ke seluruh tetangga dengan ketebalan sisi yang bervariasi, berbeda dengan proyeksi User-User yang seluruh sisi egonya setara. Empat item (ego dan tiga tetangga) saling terhubung dengan sisi tebal sehingga membentuk kelompok yang rapat, dengan sisi paling tebal terdapat antara dua item di bagian bawah. Item 274435 terhubung kuat dengan ego tetapi lemah dengan kelompok tersebut, sedangkan item 214517 hanya terhubung dengan ego dan 274435. Sisi yang tebal menunjukkan bahwa kelompok user yang mengakses kedua item hampir sama, sedangkan tidak adanya sisi menunjukkan tidak ada user yang mengakses keduanya.

6) Perbandingan proyeksi User-User dan Item-Item menunjukkan bahwa proyeksi User-User lebih efisien dari sisi ukuran graf. Proyeksi User-User memiliki 25.300 simpul dengan batas atas sisi 46.149 (rasio 0,000144 terhadap graf lengkap sebanyak 320.032.350 sisi), sedangkan proyeksi Item-Item memiliki 22.521 simpul dengan batas atas sisi 84.367 (rasio 0,000333 terhadap 253.586.460 sisi), sehingga sisi User-User sekitar 1,83 kali lebih sedikit dan grafnya lebih jarang. Kedua VSM memiliki sparsitas yang sama (99,99372%) karena yang satu adalah transpose dari yang lain, tetapi vektor Item sedikit lebih kaya informasi: 73,75% item hanya diakses satu user, sedangkan 82,79% user hanya memiliki satu item. Dari sisi waktu, satu pencarian top-k membutuhkan 0,483 ms pada User-User dan 0,446 ms pada Item-Item. Selisih sekitar 0,036 ms ini sangat kecil sehingga tidak dijadikan dasar keputusan. Berdasarkan jumlah sisi proyeksi yang lebih sedikit, proyeksi User-User dipilih sebagai yang lebih efisien.

	simpul	dimensi vektor	batas atas sisi proyeksi	sisi maks. (graf lengkap)	rasio batas atas / maks	sparsitas VSM (%)	baris dengan 1 nnz (%)	waktu 1 pencarian top-k (ms)
Proyeksi User-User	25300	22521	46149	320032350	0.000144	99.99372	82.790514	0.427990
Proyeksi Item-Item	22521	25300	84367	253586460	0.000333	99.99372	73.753386	0.399875

7) Similarity
    a. MATCH (r:User {User: $ref})-[s:SIMILAR_TO]->(n:User)
    RETURN n.visitorId AS visitorId, s.score AS cosine
    ORDER BY s.score DESC, n.visitorId ASC
    LIMIT 5
    b. User acuan dipilih bedasarkan user teraktif, yaitu 163561.

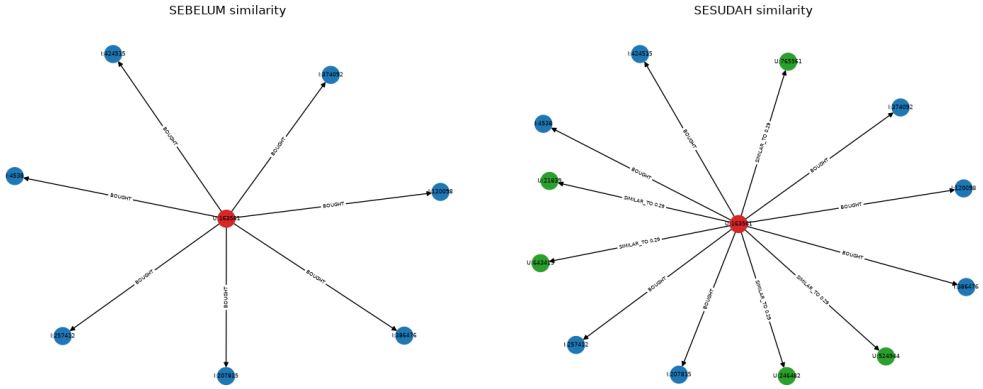
User acuan: 163561

	User	cosine
0	21839	0.287055
1	246482	0.287055
2	524944	0.287055
3	643419	0.287055
4	765561	0.287055

8) Kondisi graf sebelum dan sesudah perhitungan similarity diperlihatkan dengan query Cypher pada user acuan 163561.
    - MATCH p = (u:User {User: '163561'})-[:VIEWED|ADDED_TO_CART|BOUGHT]->(:Item) RETURN p LIMIT 7`
      Kondisi sebelum similarity ditampilkan dengan query di atas untuk mengambil relasi interaksi user acuan ke item (ditampilkan 7 dari 167 item). Hasilnya, user acuan hanya terhubung ke simpul item.
    - MATCH p = (u:User {User: '163561'})-[:SIMILAR_TO]->(:User) RETURN p
      Kondisi sesudah similarity ditampilkan dengan query yang mengambil relasi SIMILAR_TO dari user acuan ke user lain. Hasilnya, user acuan kini juga terhubung ke lima user dengan cosine tertinggi, dan setiap sisi memuat properti score dan metric.
    - MATCH ()-[r]->() RETURN type(r) AS jenis_relasi, count(r) AS jumlah ORDER BY jumlah DESC
      Query di atas digunakan untuk menghitung perubahan jumlah relasi pada seluruh graf.

    Sebelum similarity terdapat 42.890 relasi VIEWED, 1.123 ADDED_TO_CART, dan 249 BOUGHT (total 44.262, sama dengan jumlah baris P1). Sesudah similarity jumlah ketiga relasi tersebut tidak berubah, sedangkan relasi SIMILAR_TO bertambah dari 0 menjadi 5, sehingga total relasi menjadi 44.267. Dengan

demikian, penulisan similarity hanya menambah relasi baru antar-user tanpa mengubah data interaksi, sehingga graf yang semula hanya menghubungkan user dengan item kini juga menyimpan kemiripan antar-user secara eksplisit.



jenis_relasi	sebelum	sesudah
VIEWED	42890	42890
ADDED_TO_CART	1123	1123
BOUGHT	249	249
SIMILAR_TO	0	5

2.2.Analytics

Data P1 digunakan sebagai data training, yaitu matriks User x Item (matriks X) yang sudah dibentuk pada nomor 4. Data P2 dipakai sebagai ground truth, yaitu item yang benar-benar diakses user setelah periode P1. Item yang sudah diakses di P1 tidak direkomendasikan lagi, item relevan yang dipakai adalah item P2 yang belum pernah diakses user tersebut di P1 (disebut truth_new).

Diperoleh:

user di P1 dan P2: 1077

jumlah kandidat: 232

1) KNN

a. def rekomendasi_knn(idx, k=k_neighbors, n=n_rec):

tetangga, sim = top_k_similar(Xn, idx, k)

skor = np.asarray(X[tetangga].T @ sim).ravel()

skor[X[idx].indices] = 0

urut = np.argsort(-skor, kind="stable")[:n]

```
    urut = urut[skor[urut] > 0]

    return urut, skor[urut], tetangga, sim
```

Rekomendasi dibuat dengan KNN berbasis user. Pertama dicari k = 5 user dengan cosine similarity tertinggi terhadap user target. Kedua, skor setiap item dihitung dengan menjumlahkan hasil kali cosine tetangga dengan bobot interaksi tetangga pada item tersebut. Ketiga, item yang sudah pernah diakses user target di P1 dibuang. Keempat, item diurutkan dari skor terbesar dan lima teratas diambil. User dipilih secara acak dari daftar kandidat. Daftar kandidat diurutkan lebih dulu, kemudian `np.random.default_rng(seed)` dipakai untuk memilih satu posisi, sehingga hasilnya sama setiap kali kode dijalankan. Selain punya minimal dua item di P1 dan minimal satu item baru di P2, kandidat juga harus menghasilkan minimal tiga rekomendasi dari KNN, supaya tiga item iklan benar-benar berasal dari KNN.

Dari 1.077 user yang muncul di P1 dan P2, terdapat 353 user dengan minimal dua item di P1, dan 232 di antaranya memiliki setidaknya satu item baru di P2. Dari 232 user tersebut, 95 user menghasilkan minimal tiga rekomendasi dari KNN. Dari daftar 95 user yang sudah diurutkan ini, np.random.default_rng(42) memilih user 1135150, yang memiliki 3 item di P1 dan 1 item baru di P2. Karena seed dan urutan daftar tidak berubah, user yang terpilih akan selalu sama setiap notebook dijalankan ulang. Skrip KNN terdapat pada fungsi rekomendasi_knn, yang mengikuti empat langkah yang dijelaskan di atas.

b. User 1135150 memiliki lima tetangga terdekat, yaitu user 612331 dan 740987 dengan cosine 0,5345, user 1044457 dengan cosine 0,3780, user 1334951 dengan cosine 0,3086, dan user 82131 dengan cosine 0,2965. Dari item yang diakses kelima tetangga tersebut, dan yang belum pernah diakses user 1135150, diperoleh lima item dengan skor tertinggi, yaitu 347414 (0,6745), 279052 (0,5930), 135900 (0,3086), 309778 (0,3086), dan 136251 (0,2965). Maka tiga item yang diiklankan untuk k = 5 adalah item 347414, 279052, dan 135900. Item 135900 dan 309778 memiliki skor yang sama sehingga urutannya ditentukan oleh urutan indeks item.

	tetangga	cosine
0	612331	0.534522
1	740987	0.534522
2	1044457	0.377964
3	1334951	0.308607
4	82131	0.296500

	peringkat	item	skor
0	1	347414	0.674464
1	2	279052	0.592999
2	3	135900	0.308607
3	4	309778	0.308607
4	5	136251	0.296500

2) Hit@5 dan NDCG@5
   a. Hit@5

      Hit@5 bernilai 1 bila ada minimal satu item relevan di antara lima item rekomendasi, dan bernilai 0 bila tidak ada. Item relevan adalah item yang diakses user di P2 tetapi belum pernah diakses di P1 (truth_new).

	peringkat	item	relevan
0	1	347414	0
1	2	279052	0
2	3	135900	0
3	4	309778	0
4	5	136251	0

```
item relevan di P2: ['452852']
Hit@5 = 0
```

      Item relevan untuk user 1135150 hanya satu, yaitu item 452852, yang diakses di P2 tetapi belum pernah diakses di P1. Item ini tidak termasuk dalam lima rekomendasi (347414, 279052, 135900, 309778, dan 136251), sehingga tidak ada item relevan yang berhasil ditemukan. Dengan demikian Hit@5 untuk user ini bernilai 0.

   b. NDCG@5

      NDCG@5 memperhitungkan posisi item relevan. DCG dihitung dengan menjumlahkan 1 / log2(i + 1) untuk setiap item relevan di peringkat i. IDCG adalah DCG ideal, yaitu bila semua item relevan user (paling banyak 5) berada di peringkat teratas. NDCG@5 adalah DCG dibagi IDCG. Tetapi karena tidak ada item relevan di lima rekomendasi, DCG bernilai 0. User ini hanya memiliki satu item relevan, sehingga IDCG dihitung dari satu item di peringkat pertama, yaitu 1 / log2(2) = 1. Dengan demikian NDCG@5 = 0 / 1 = 0.

```
DCG@5  = 0
IDCG@5 = 1.0
NDCG@5 = 0.0
```

Nilai Hit@5 dan NDCG@5 sebesar 0 pada user 1135150 tidak otomatis berarti model tidak bekerja, karena satu user acak hanya satu sampel. Sebagai gambaran, pada evaluasi tambahan terhadap 232 user kandidat, rata-rata Hit@5 untuk User-KNN adalah 0,0388 dan rata-rata NDCG@5 adalah 0,0153, hanya sedikit di atas baseline popularitas (0,0345 dan 0,0140) dan selisihnya masih berada dalam standard error. Nilai yang rendah ini berkaitan dengan data yang sangat jarang. Sebanyak 82,79% user hanya memiliki satu item di P1, dan KNN hanya menghasilkan minimal satu rekomendasi untuk 57,3% user kandidat, sehingga untuk banyak user tidak ada tetangga yang membawa item baru. Karena itu hasil user acak sebaiknya dilaporkan apa adanya, disertai rata-rata pada seluruh user kandidat.